A single distribution or comparison of multiple distributions: Penguins Data Visualization

XU Shirong

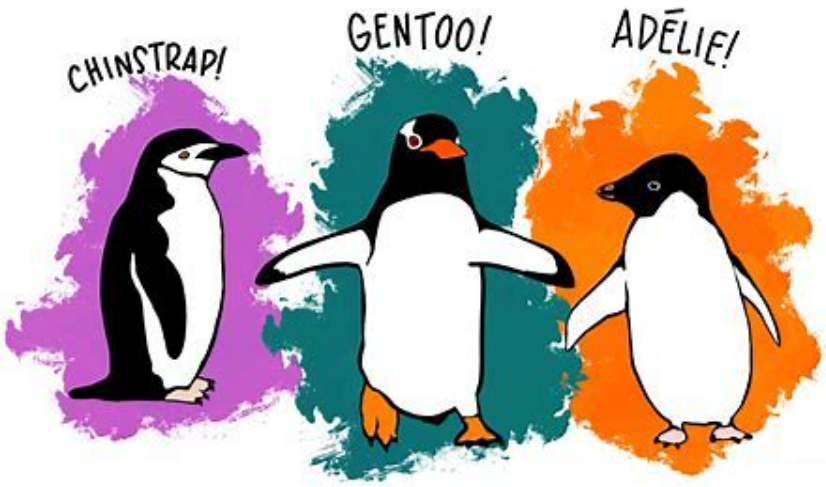
Agenda

- Visualizing distributions
- Single vs multiple distributions
- Histograms & Density plots
- Boxplots, Violin plots, Raincloud plots

Penguins Dataset Overview

- Categorical: species, island, sex
- Numeric: bill length, depth, flipper length, body mass
- Ordinal: year

Goal: Visualize distributions and differences by sex and species

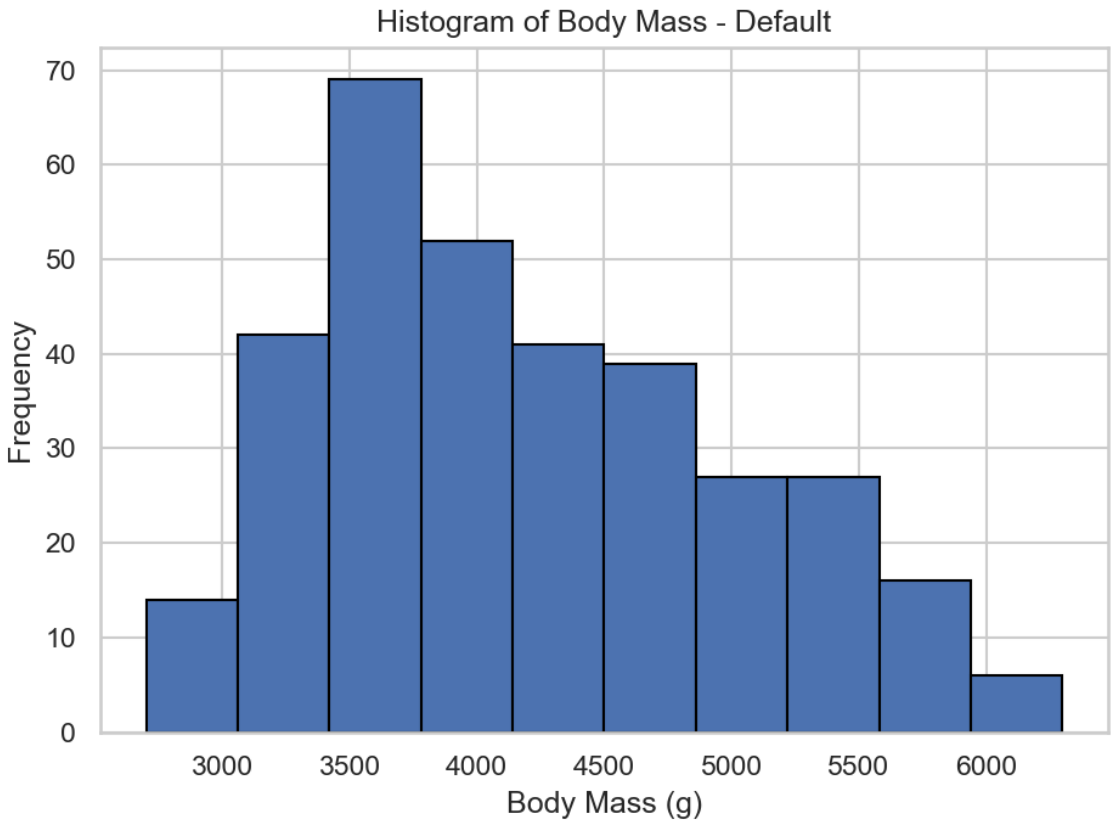


```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from palmerpenguins import load_penguins
from scipy.stats import gaussian_kde
from matplotlib.patches import Rectangle
penguins = load_penguins().dropna()
```

Question: Does this dataset contain NA?

Histogram of Body Mass - Default

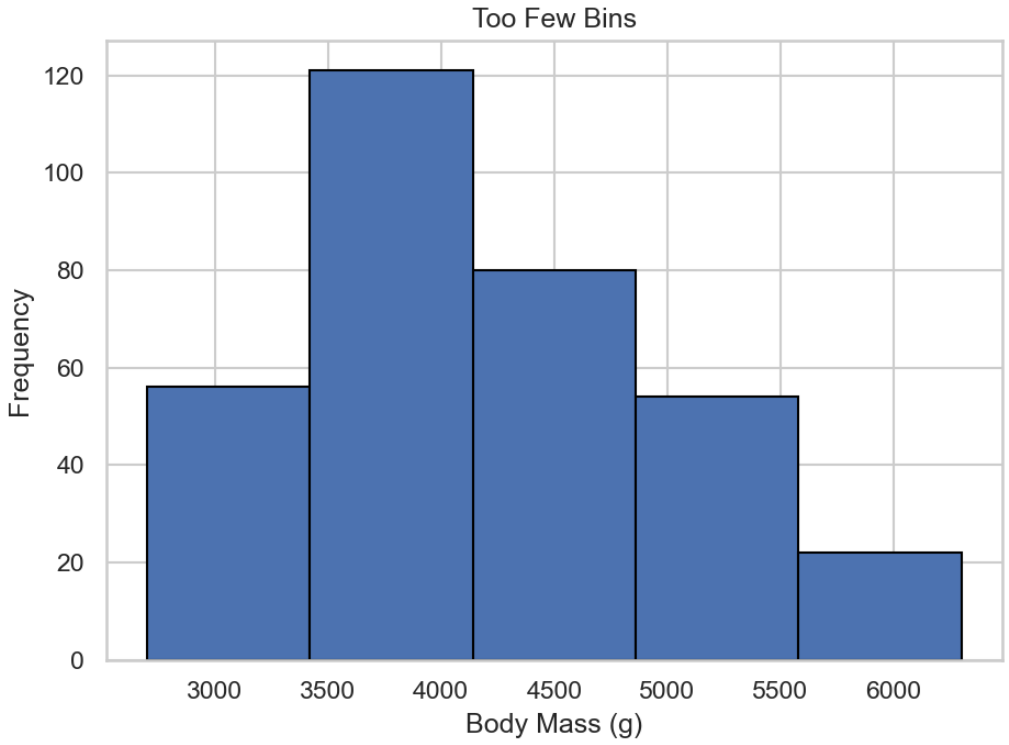
```
plt.hist(penguins['body_mass_g'], edgecolor='black')
plt.title('Histogram of Body Mass - Default')
```



Interpretation: Default histogram shows the overall distribution of body mass. Q: Can we change the histogram?

Too Few Bins

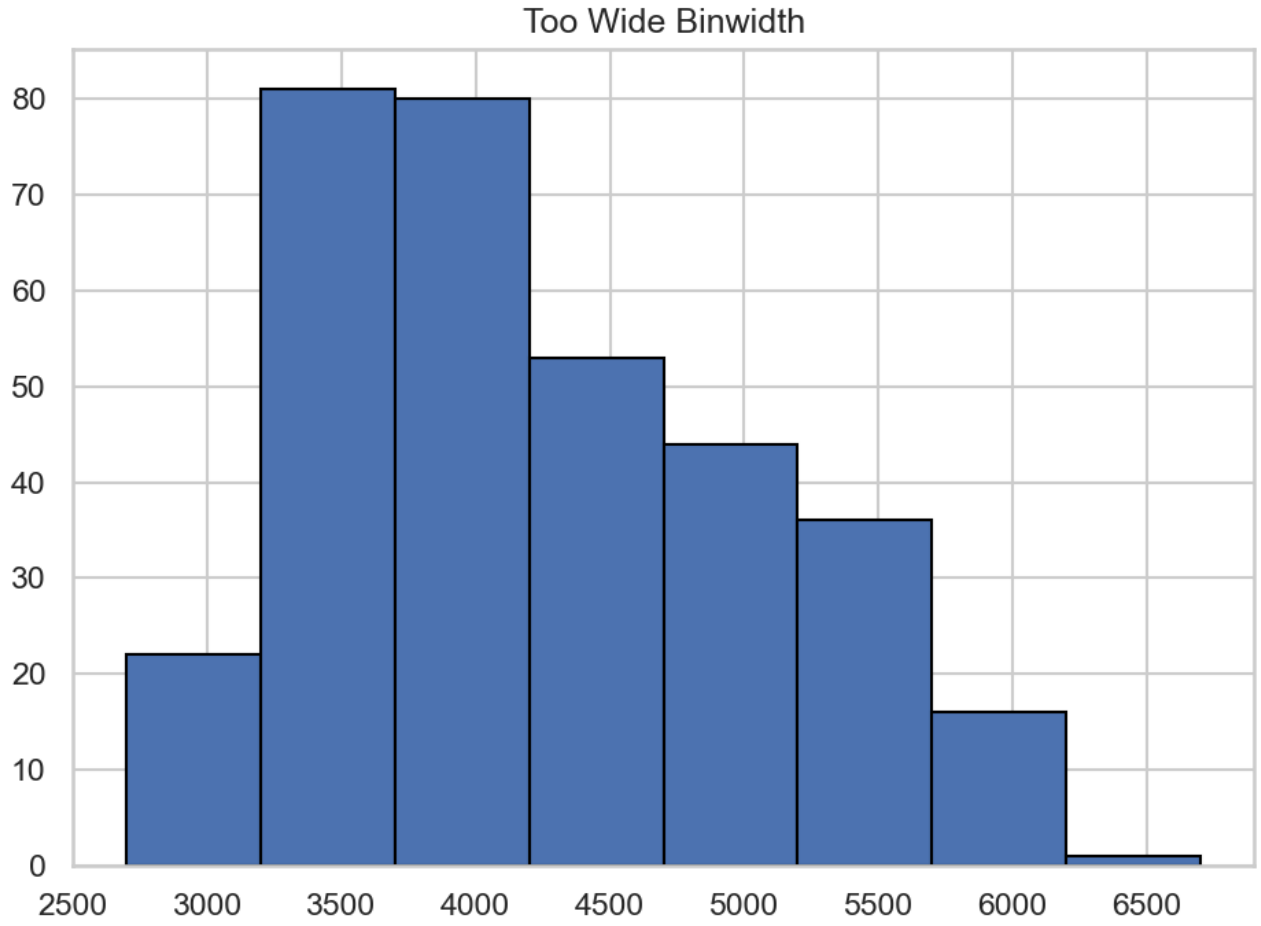
```
plt.hist(penguins['body_mass_g'], bins=5, edgecolor='
    black')
plt.title('Too␣Few␣Bins')
```



Interpretation: Too few bins obscure distribution details; difficult to identify modes.

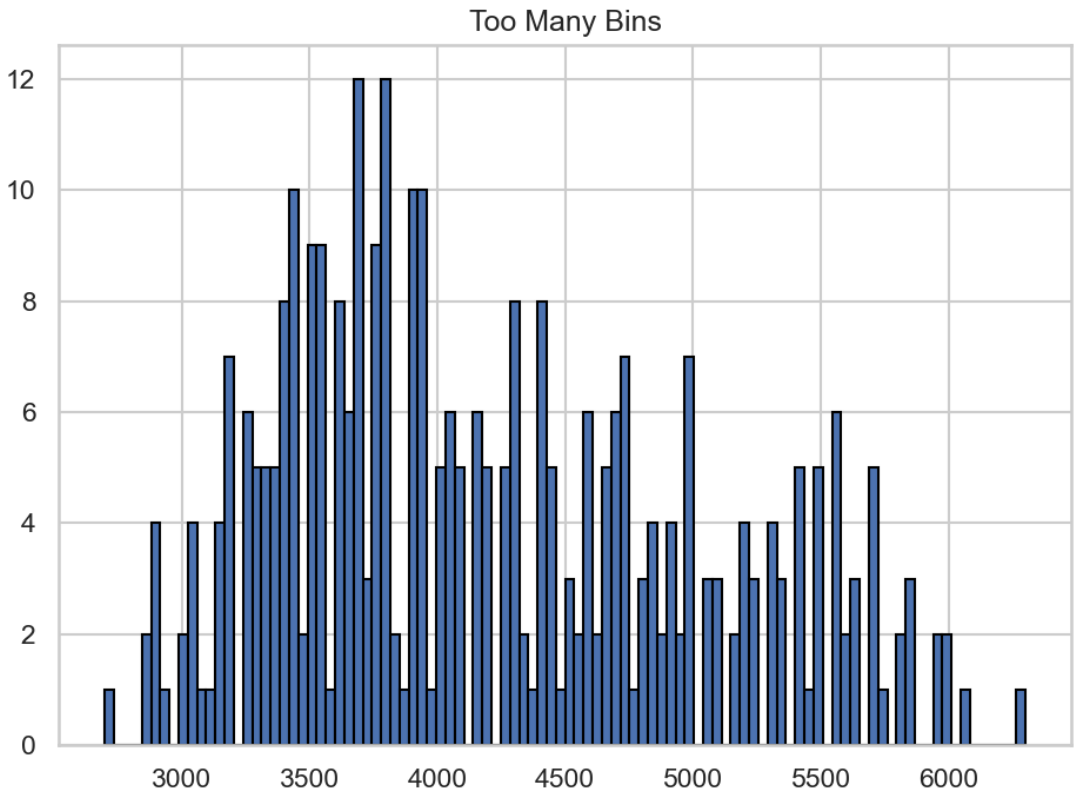
Too Wide Binwidth

```
bins = np.arange(data.min(), data.max() + 500, 500)
plt.hist(data, bins=bins, edgecolor='black')
plt.title('Too Wide Binwidth')
```



Too Many Bins

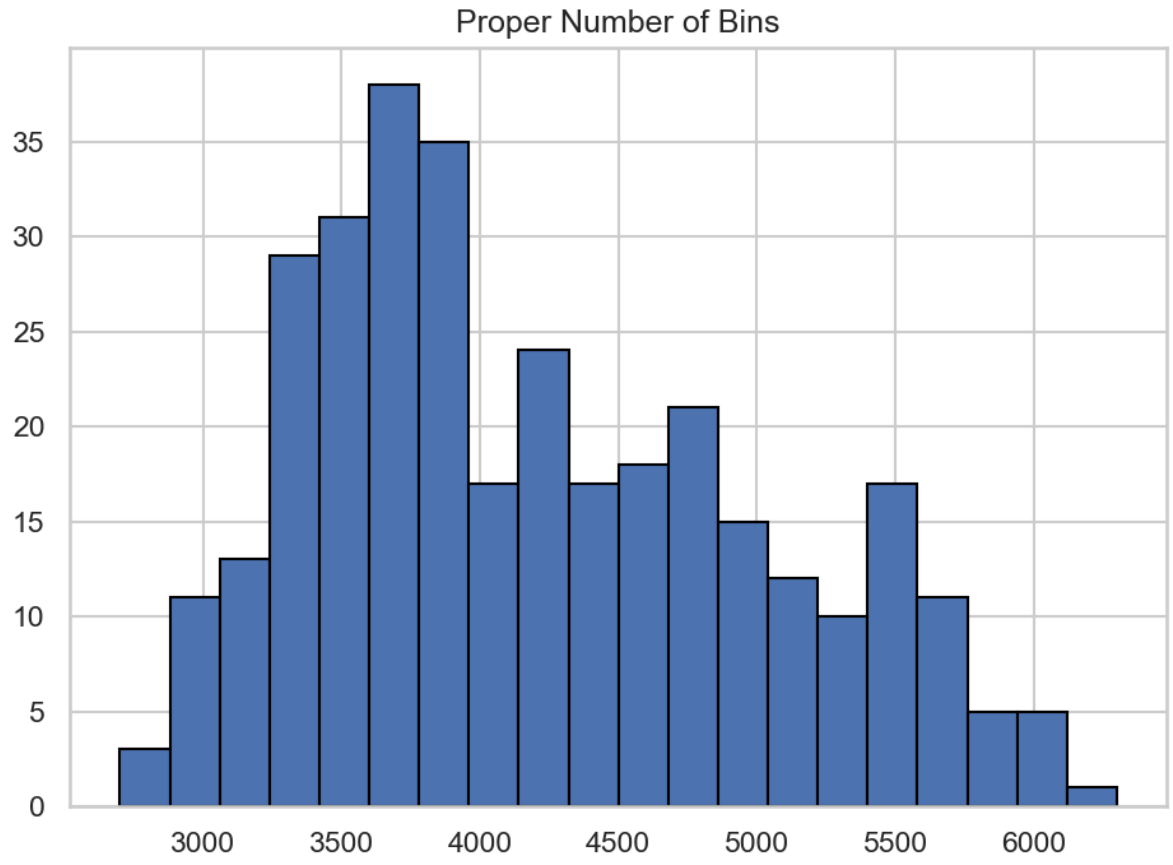
```
plt.hist(data, bins=100, edgecolor='black')
plt.title('Too Many Bins')
```



Interpretation: Too many bins increase noise and obscure trends.

Proper Number of Bins

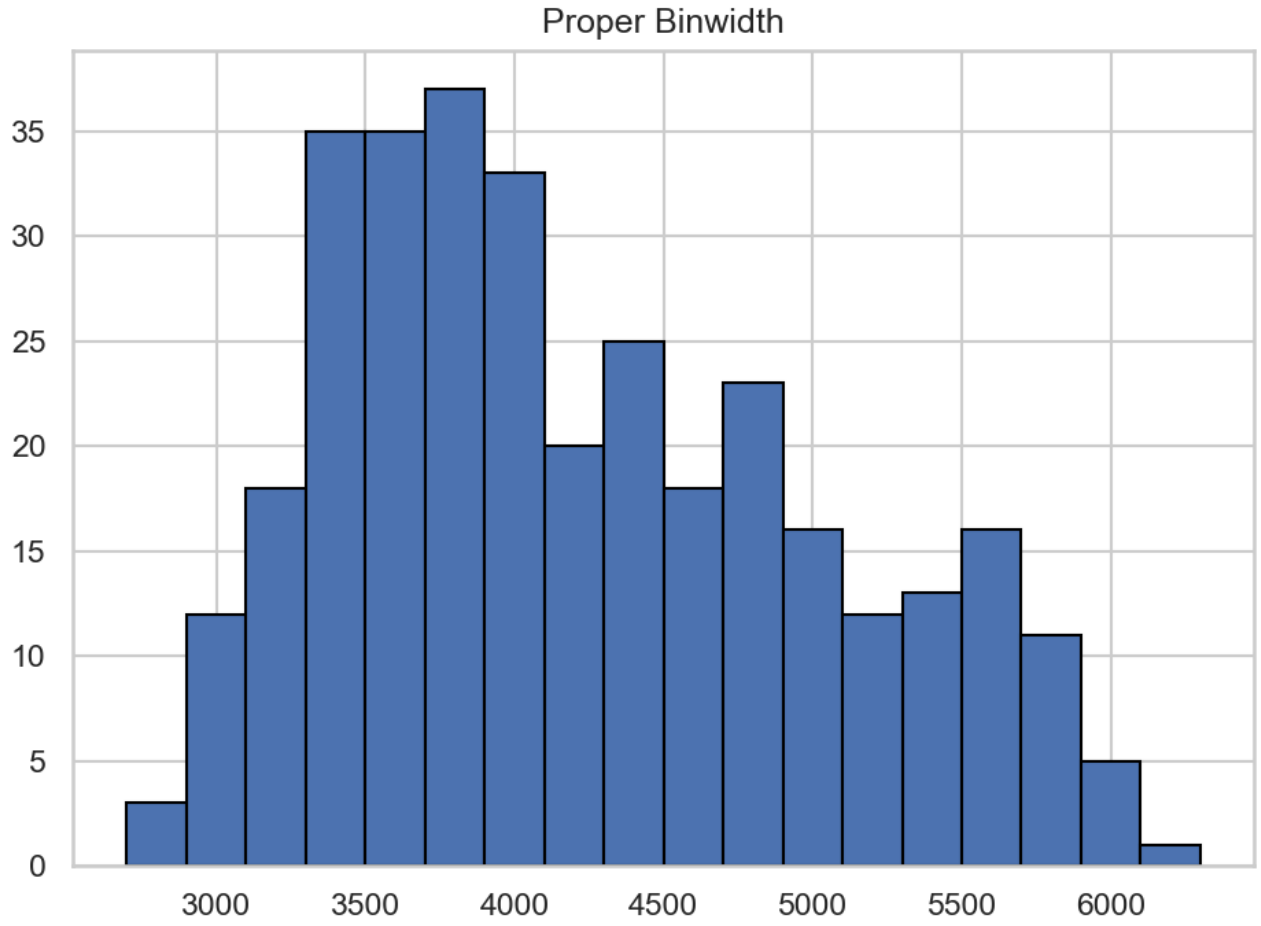
```
plt.hist(data, bins=20, edgecolor='black')
plt.title('Proper Number of Bins')
```



Interpretation: Proper bin number balances detail and readability.

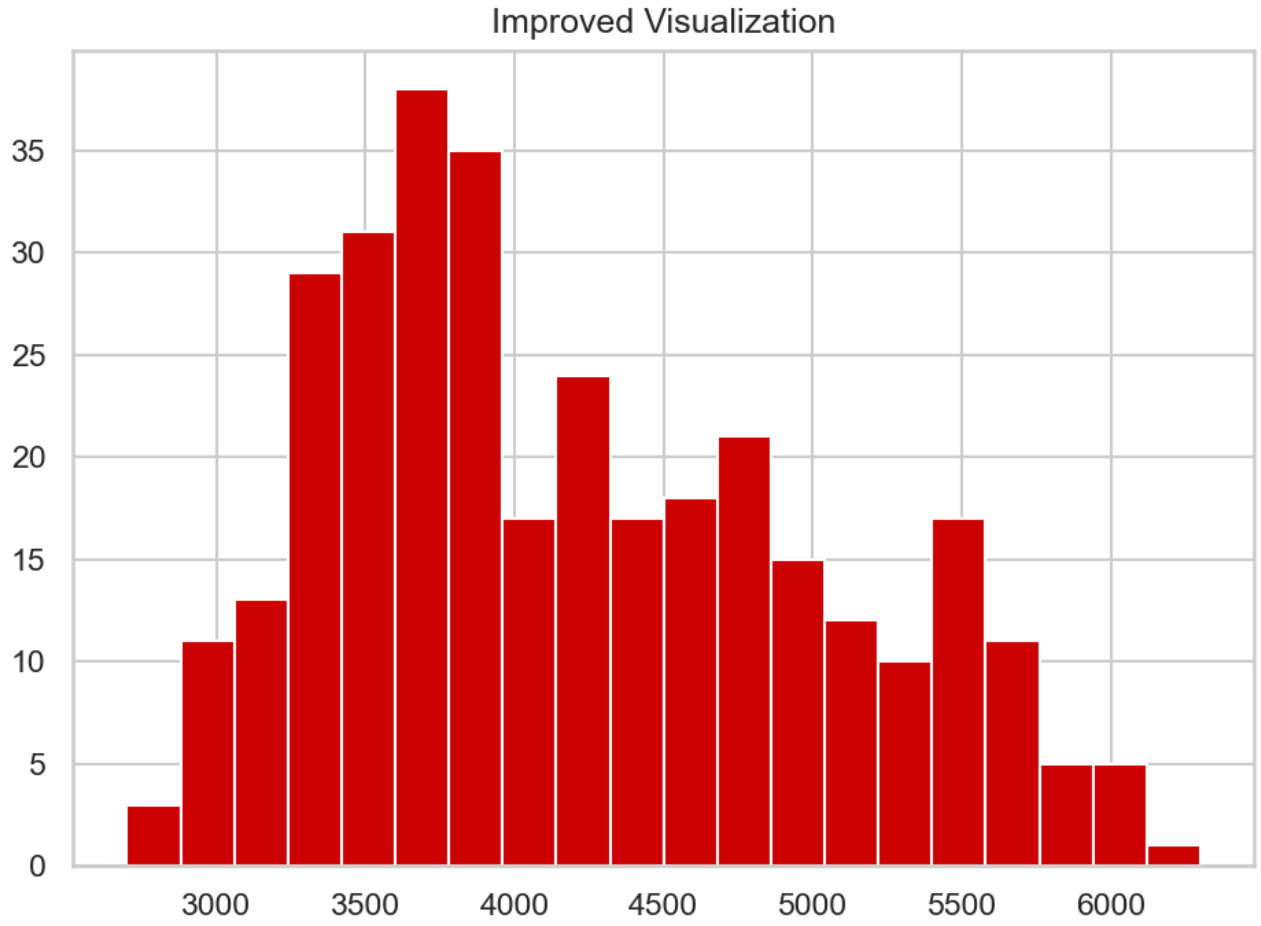
Proper Binwidth

```
bins = np.arange(data.min(), data.max() + 200, 200)
plt.hist(data, bins=bins, edgecolor='black')
plt.title('Proper Binwidth')
```



Improved Visualization

```
plt.hist(data, bins=20, color='#CC0000', edgecolor='
    white')
plt.title('Improved␣Visualization')
```

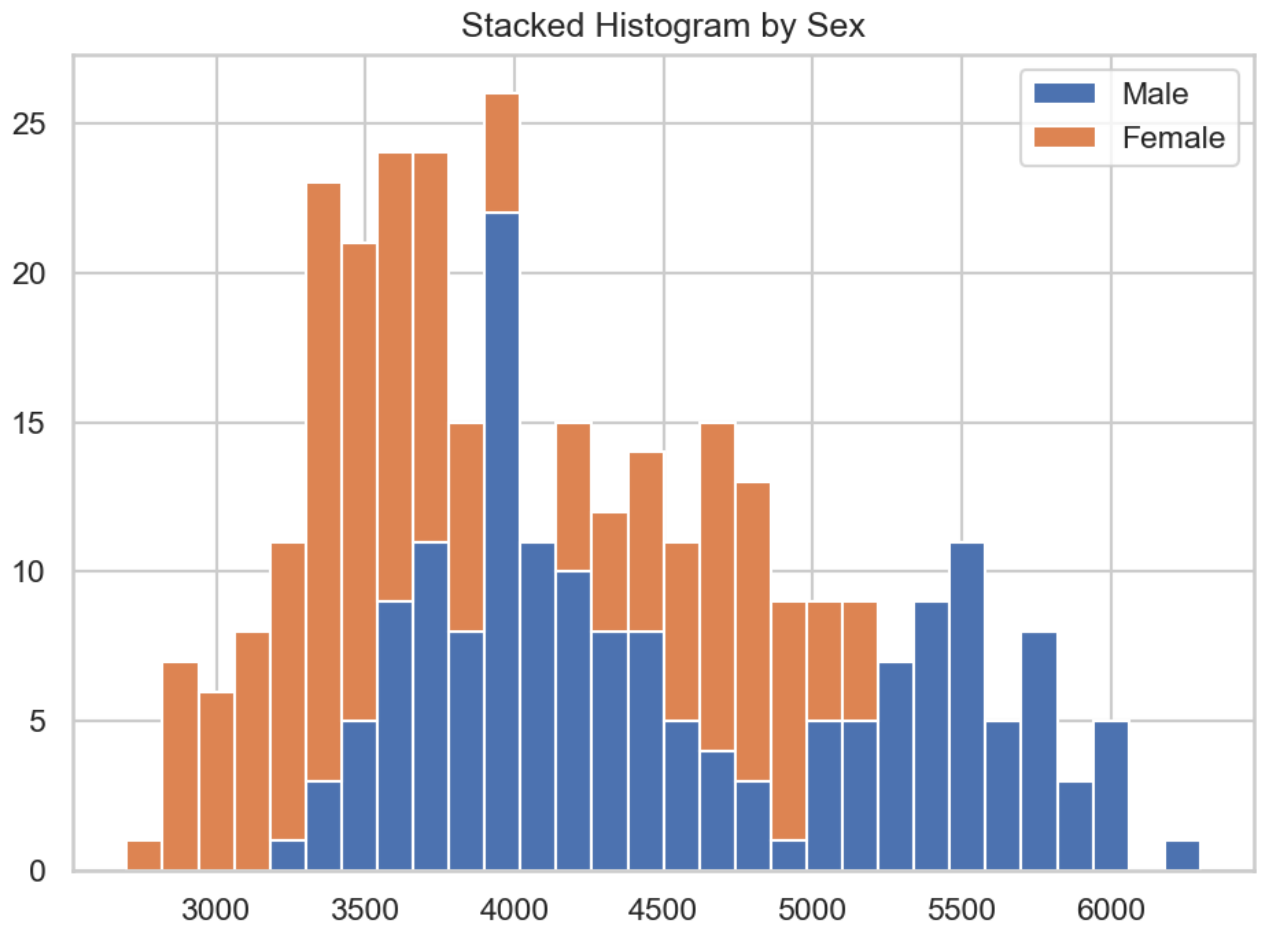


Stacked Histogram by Sex

```
male = penguins[penguins['sex'] == 'male']['
    body_mass_g']
female = penguins[penguins['sex'] == 'female']['
    body_mass_g']
plt.hist([male, female], bins=30, stacked=True,
         label=['Male','Female'], edgecolor='white')
plt.legend()
plt.title('Stacked Histogram by Sex')
```

We use conditional dataframe and plot male and female mass distributions.

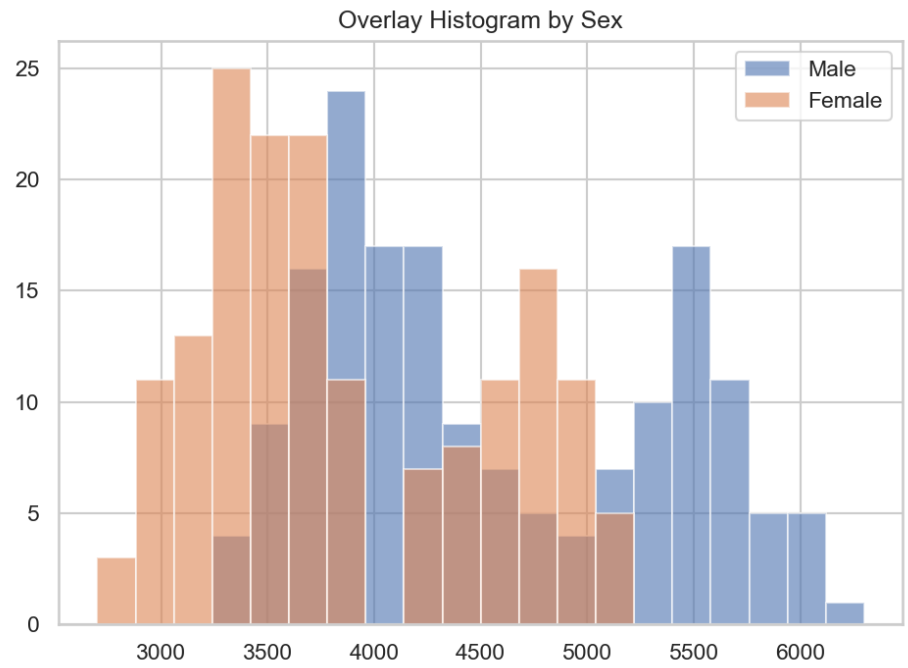
Stacked Histogram by Sex



Interpretation: Stacked histograms show male vs female contribution to total body mass. Peaks and proportions are visible. Question: What is the main problem of this plot?

Overlay Histogram by Sex

```
plt.hist(male, bins=bins, alpha=0.6, label='Male')
plt.hist(female, bins=bins, alpha=0.6, label='Female')
plt.legend()
plt.title('Overlay Histogram by Sex')
```



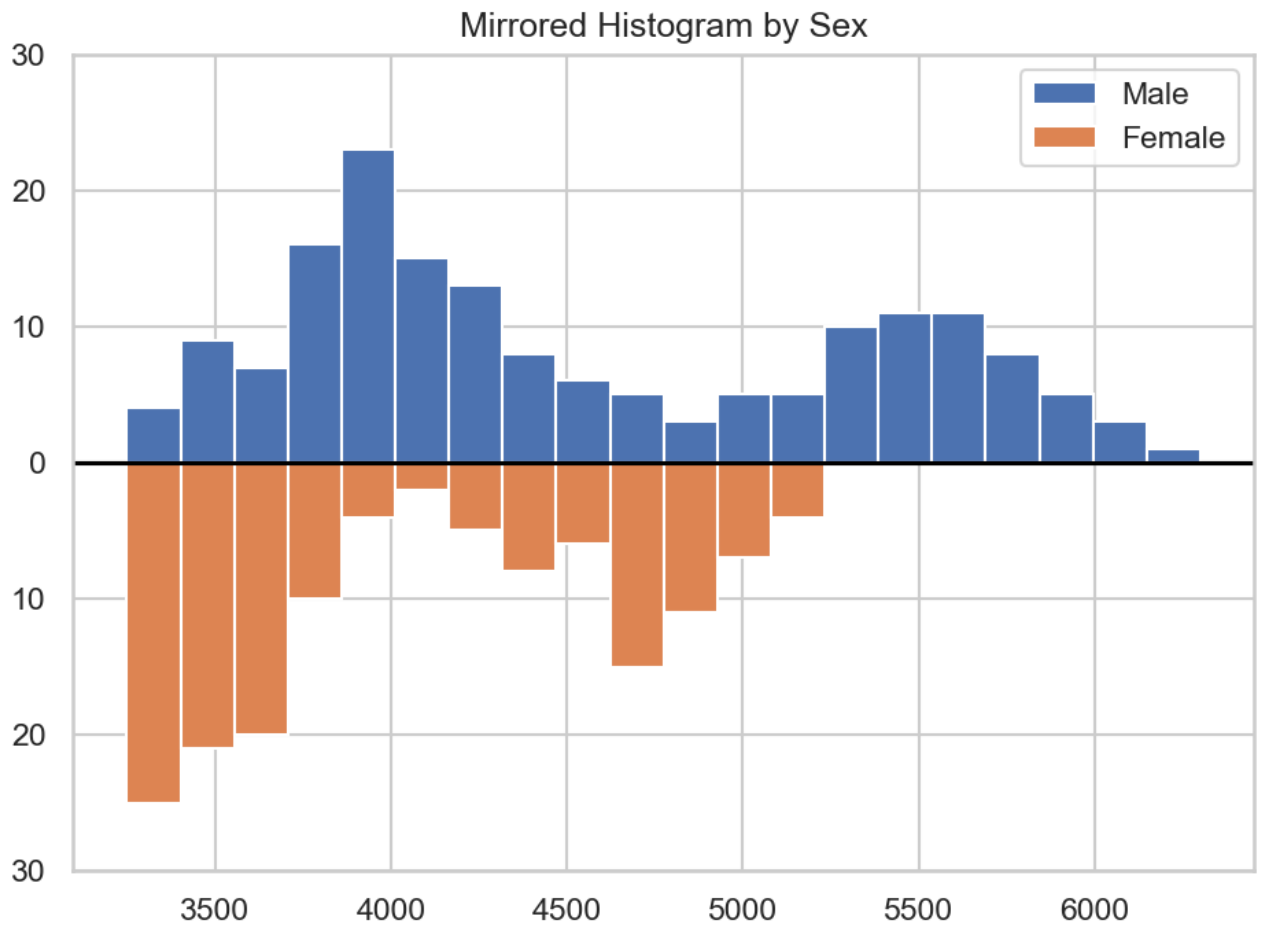
Interpretation: Overlay histograms allow direct comparison of male and female distributions.

Mirrored Histogram by Sex

```
counts_m, bins_m = np.histogram(male, bins=20)
counts_f, _ = np.histogram(female, bins=bins_m)
centers = 0.5 * (bins_m[:-1] + bins_m[1:])
widths = np.diff(bins_m)
plt.figure()
plt.bar(centers, counts_m, width=widths, label='Male')
plt.bar(centers, -counts_f, width=widths, label='
    Female')
plt.axhline(0, color='black')
plt.yticks(plt.yticks()[0], [abs(int(x)) for x in plt.
    yticks()[0]])
plt.legend()
plt.title('Mirrored Histogram by Sex')
```

Hint: We have to use the same bins.

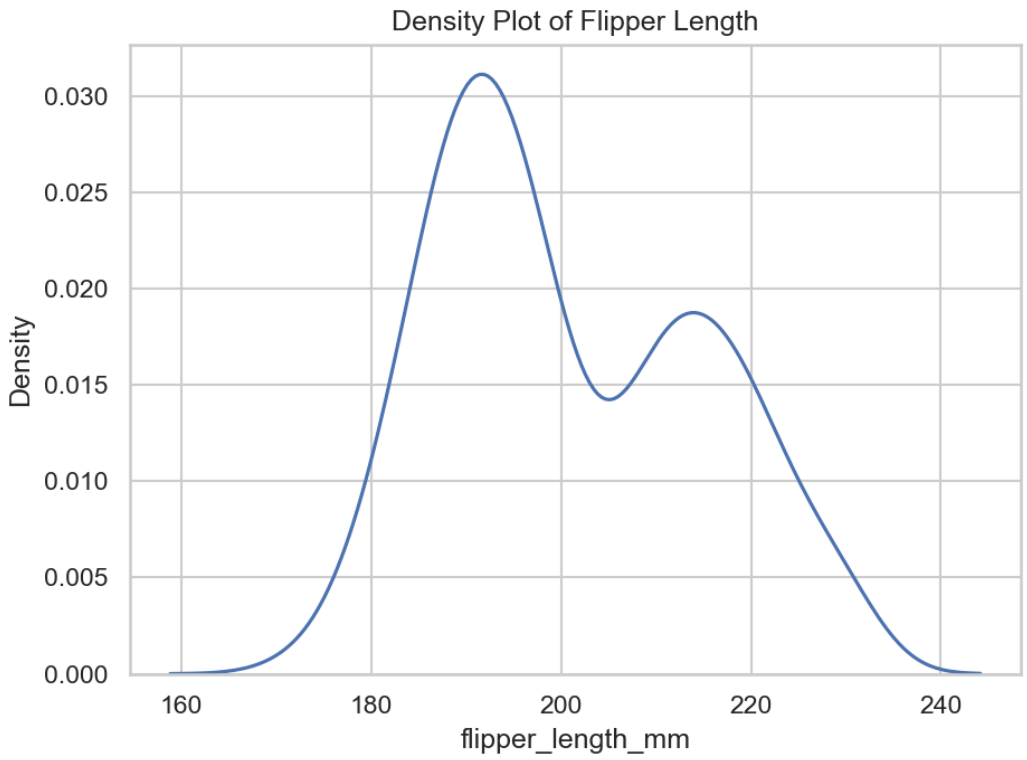
Mirrored Histogram by Sex



Interpretation: Mirrored histogram visualizes male and female distributions symmetrically, making comparison intuitive.

Density Plot of Flipper Length

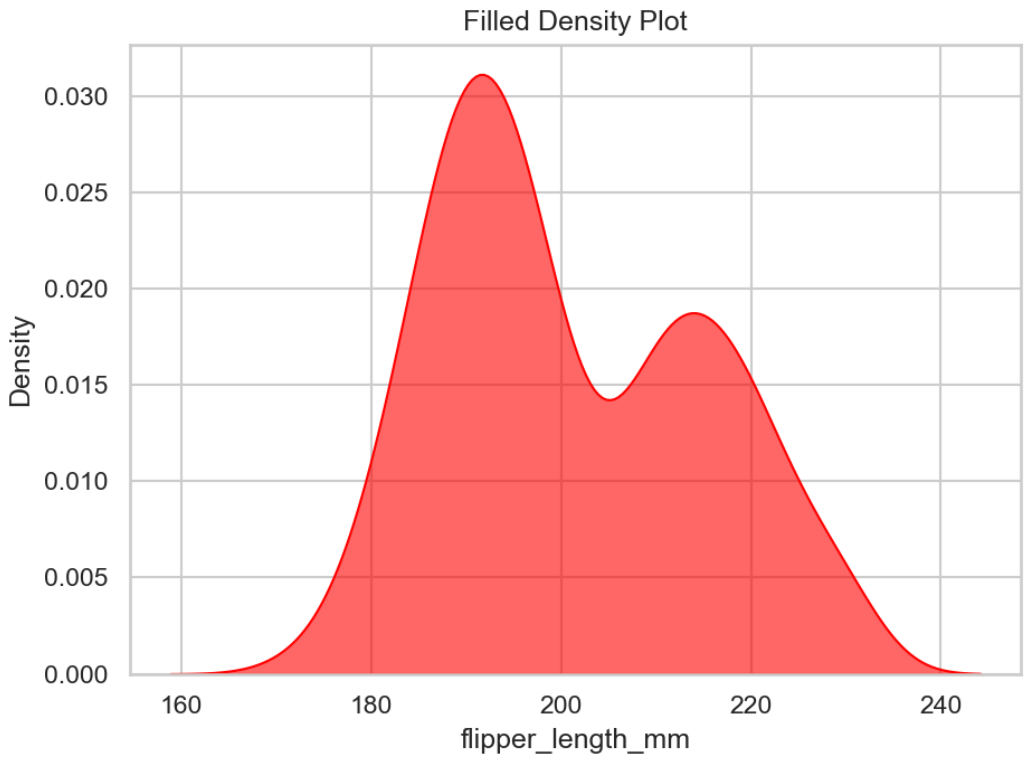
```
sns.kdeplot(data=penguins, x='flipper_length_mm')
plt.title("Density Plot of Flipper Length")
```



Interpretation: Kernel density estimate smooths the distribution, revealing the shape of flipper length data.

Filled Density Plot

```
sns.kdeplot(data=penguins, x='flipper_length_mm', fill
    =True, color='red', alpha=0.6)
plt.title("Filled Density Plot")
```



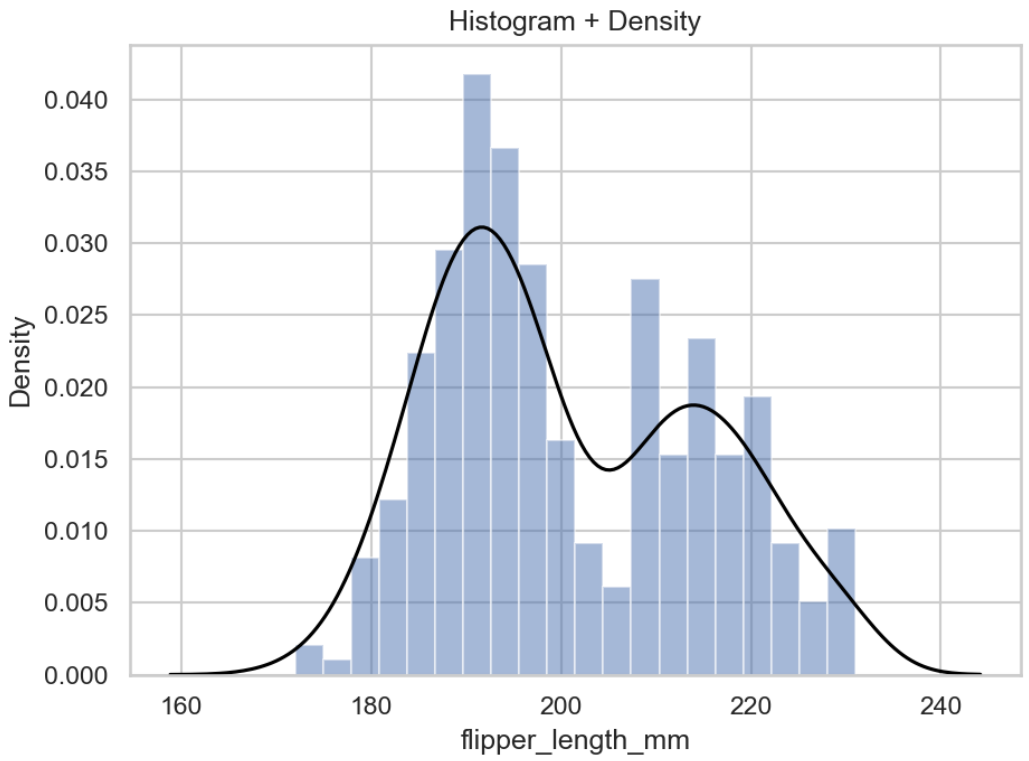
Interpretation: Filling the area under the curve emphasizes the density visually.

Histogram + Density

```
plt.hist(penguins['flipper_length_mm'], bins=20,
    density=True, alpha=0.5)
sns.kdeplot(penguins['flipper_length_mm'], color='
    black')
plt.title("Histogram + Density")
```

Hint: For plt.hist, set density=True.

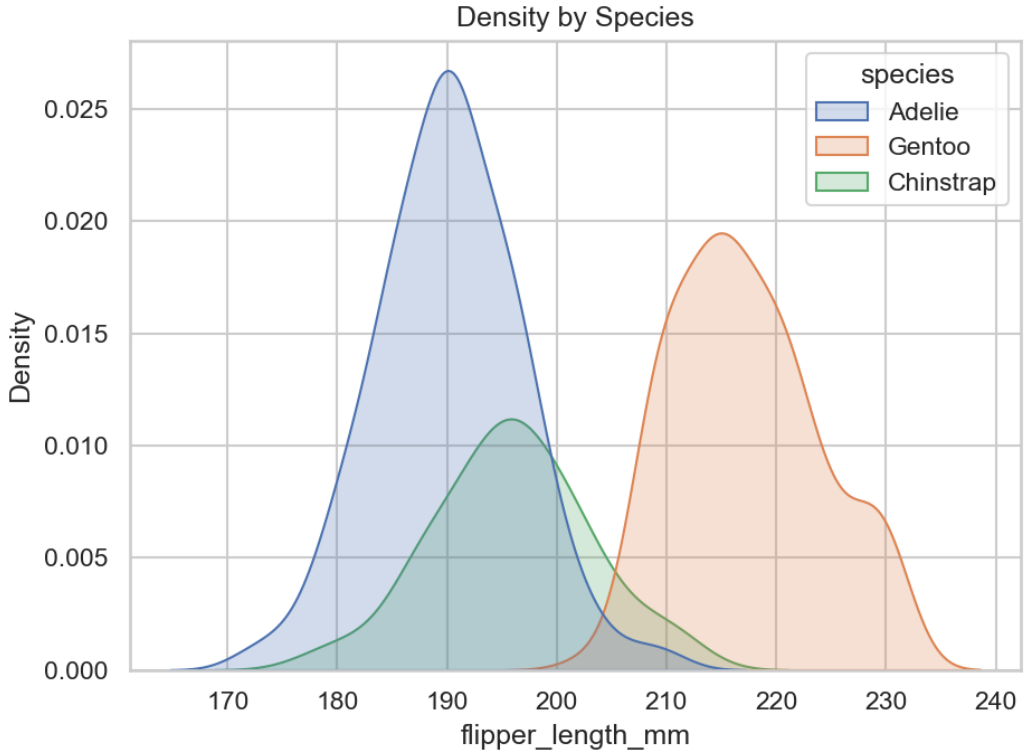
Histogram + Density



Interpretation: Overlaying histogram and density shows raw counts and smooth distribution simultaneously.

Density by Species

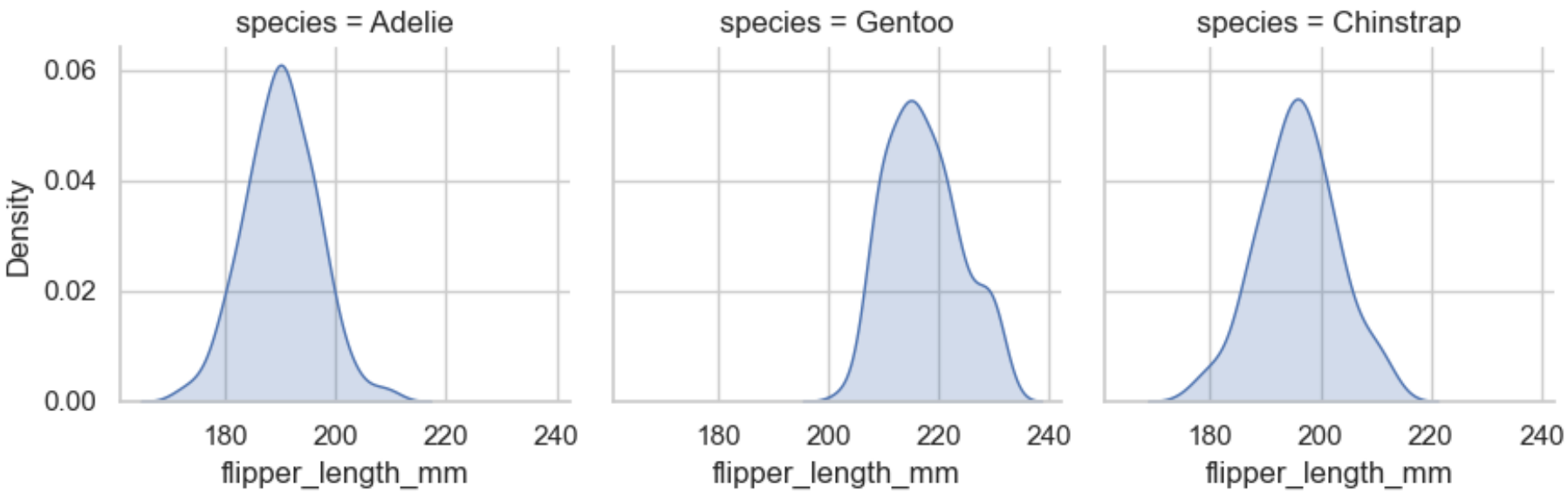
```
sns.kdeplot(data=penguins, x='flipper_length_mm', hue=
    'species', fill=True)
plt.title("Density by Species")
```



Interpretation: Comparing species reveals differences in flipper length distributions.

FacetGrid KDE by Species

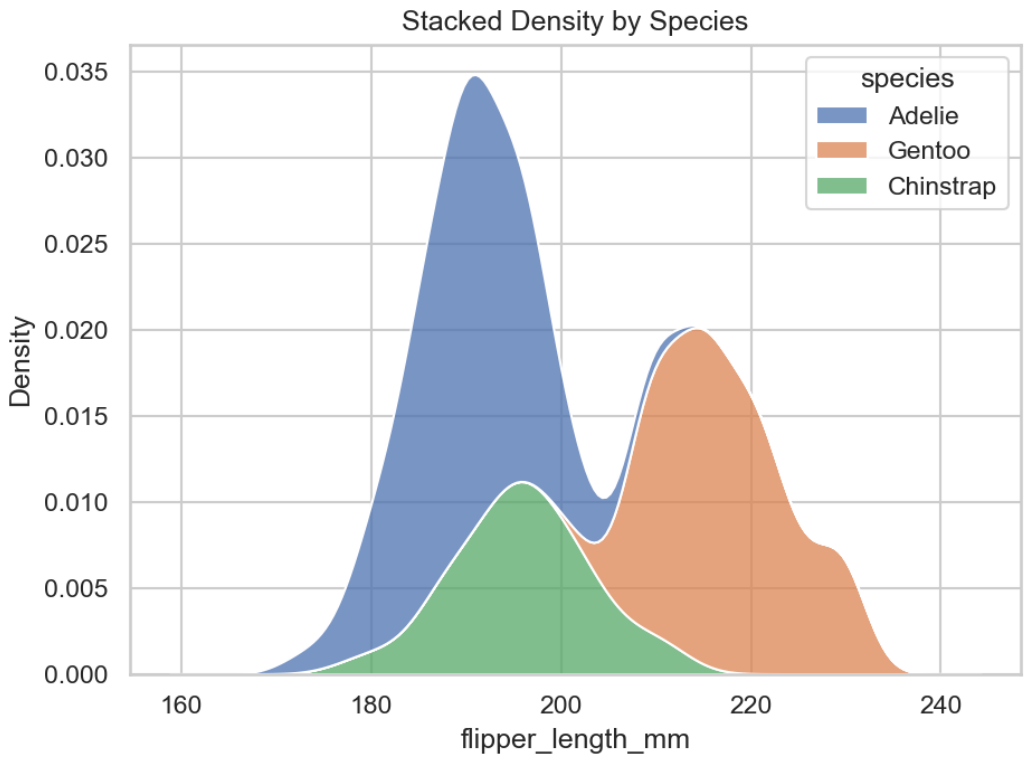
```
g = sns.FacetGrid(penguins, col='species')
g.map(sns.kdeplot, 'flipper_length_mm', fill=True)
```



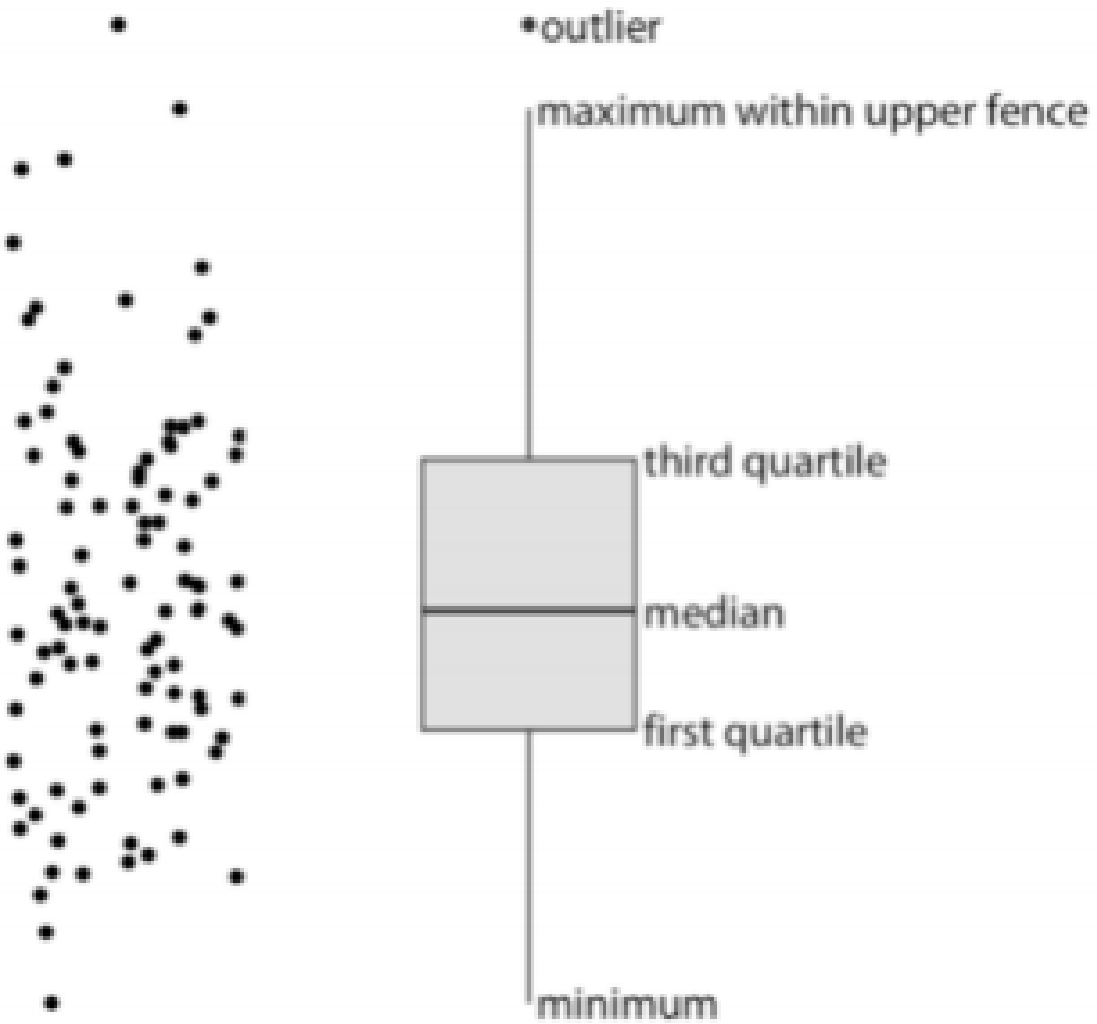
Interpretation: Separate plots for each species make patterns clearer.

Stacked Density by Species

```
sns.kdeplot(data=penguins, x='flipper_length_mm', hue=
    'species', multiple='stack', fill=True)
plt.title("Stacked Density by Species")
```

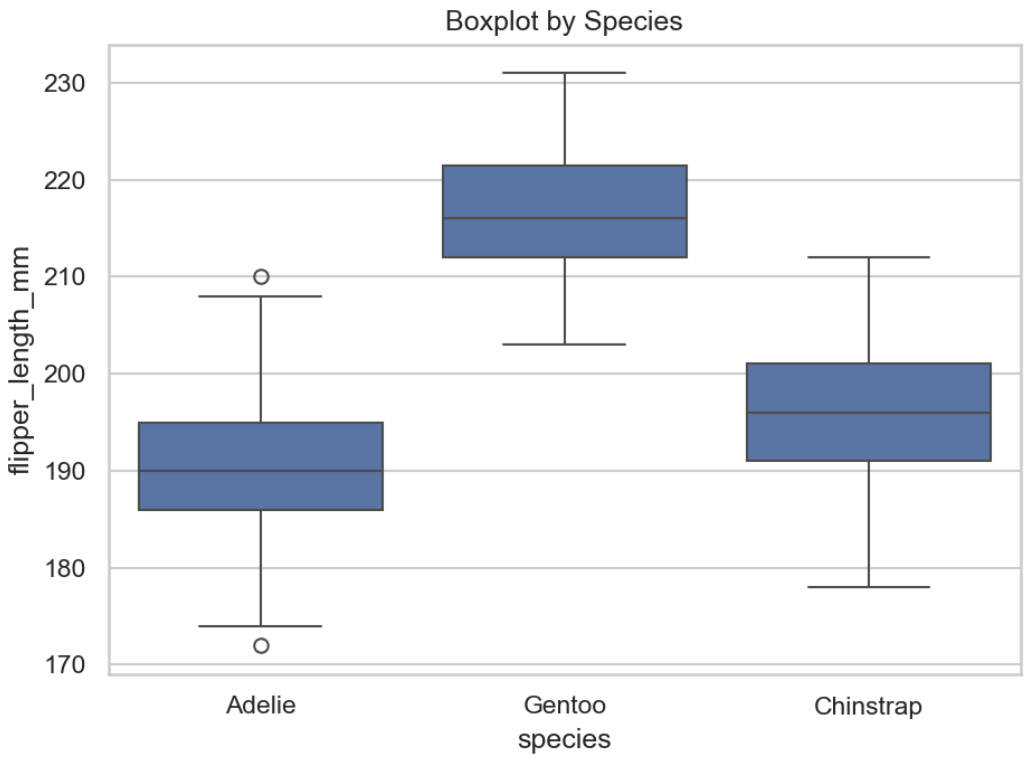


Interpretation: Stacked densities highlight total distribution and contribution from each species.


outlier
maximum within upper fence
third quartile
median
first quartile
minimum

Boxplot by Species

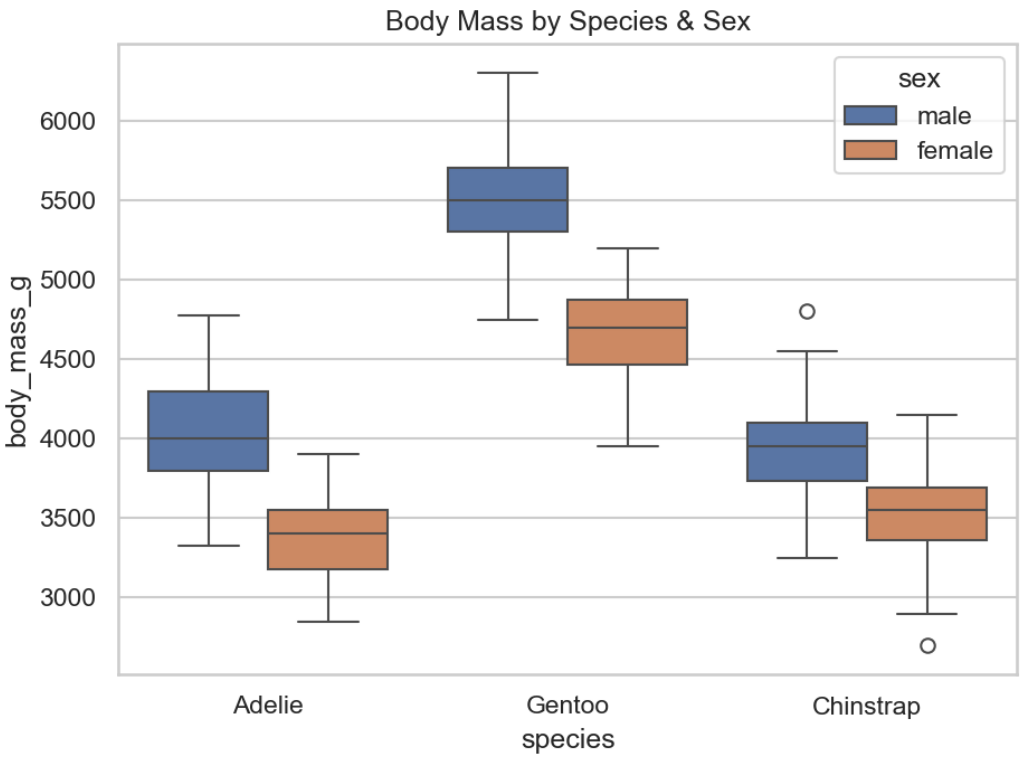
```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm')
plt.title("Boxplot by Species")
```



Interpretation: Boxplot summarizes median, quartiles, and spread for each species.

Body Mass by Species & Sex

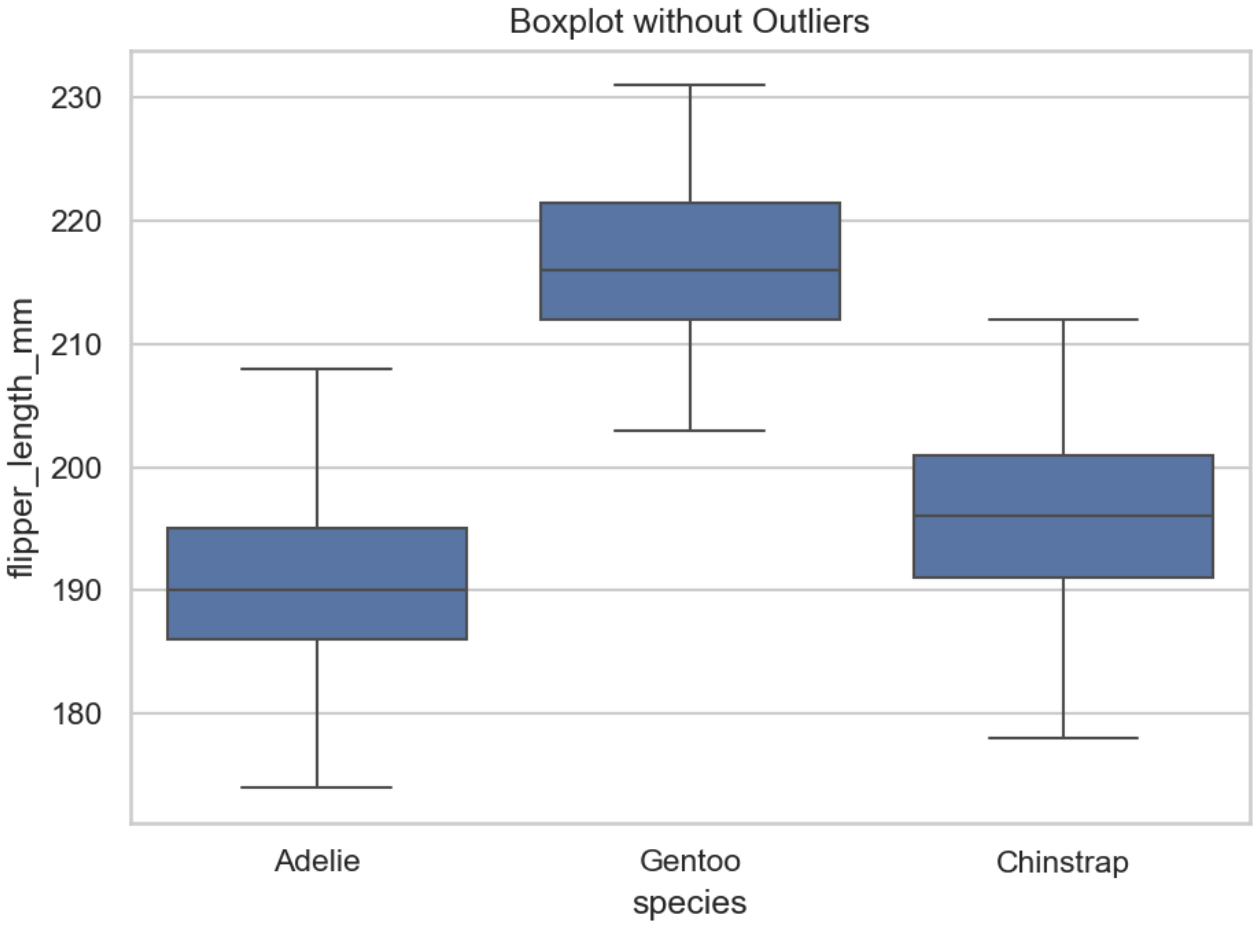
```
sns.boxplot(data=penguins, x='species', y='body_mass_g
    ', hue='sex')
plt.title("Body Mass by Species & Sex")
```



Interpretation: Boxplot grouped by species and sex shows distribution differences between male and female penguins.

Boxplot without Outliers

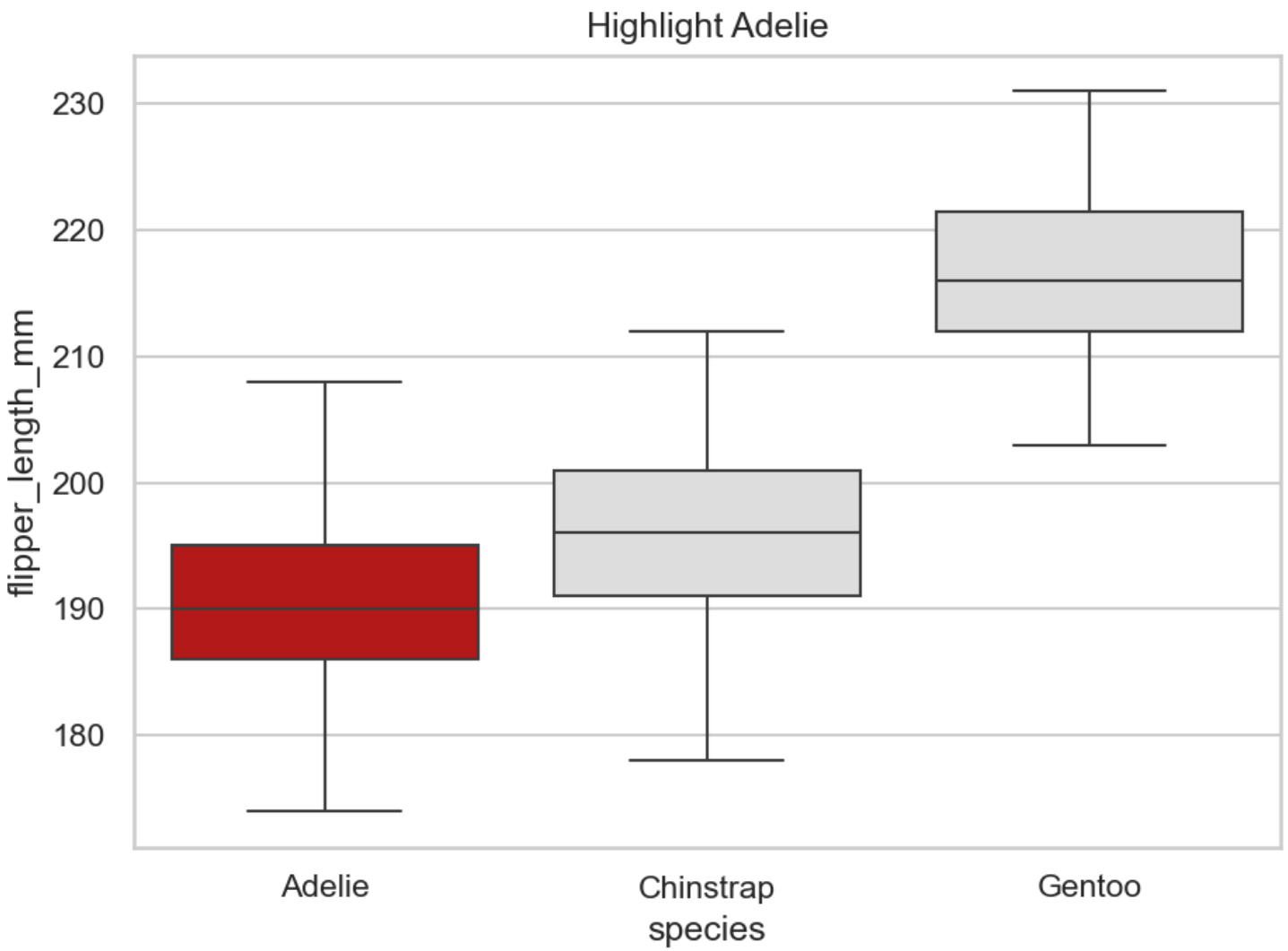
```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm', showfliers=False)
plt.title("Boxplot without Outliers")
```



Highlight Adelie

```
highlight = 'Adelie'
order = sorted(penguins['species'].unique())
colors = ['#CC0000' if s == highlight else '#DDDDDD'
    for s in order]
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm',
            order=order, palette=colors, showfliers=
                False)
plt.title("Highlight␣Adelie")
```

Highlight Adelie

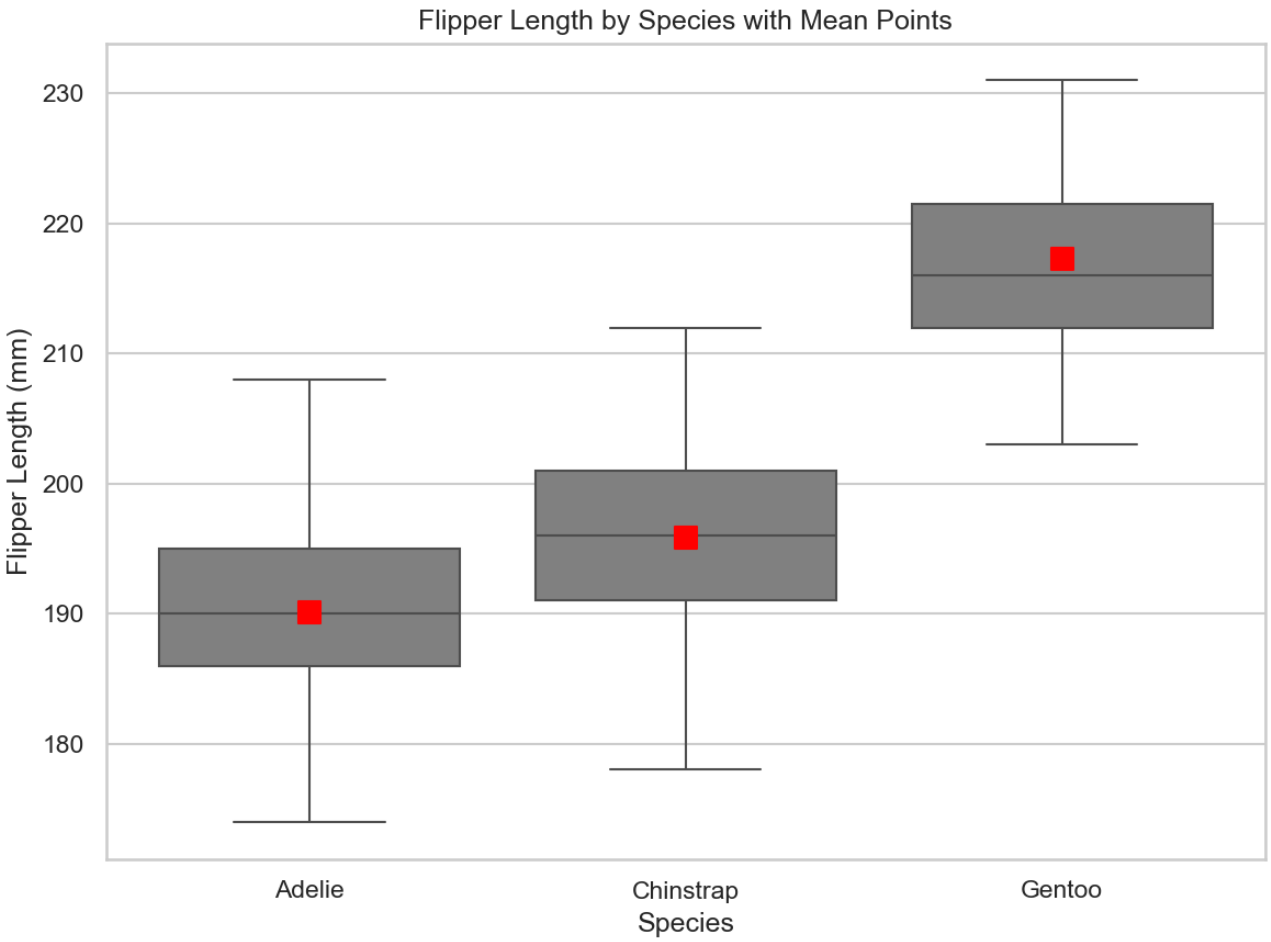


Interpretation: Highlighting one species draws attention to its distribution relative to others.

Boxplot with Mean

```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm',
                color='white', showfliers=False)
means = penguins.groupby('species')['flipper_length_mm
    '].mean()
plt.scatter(range(len(means)), means, color='red',
    marker='s', s=80)
plt.title("Boxplot␣with␣Mean")
```

Boxplot with Mean



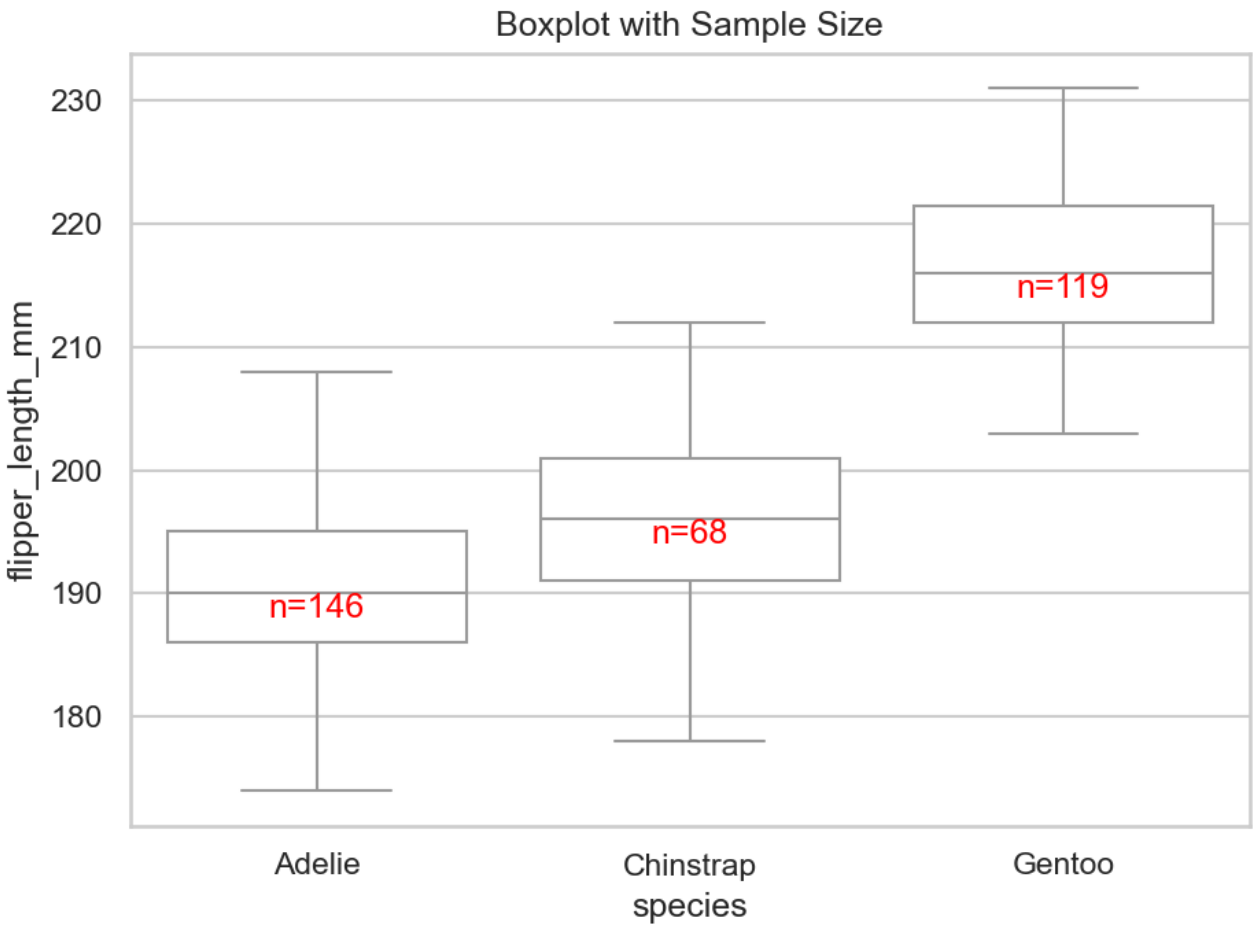
Interpretation: Overlaying mean values helps compare central tendencies with medians. Means are close to median.

Boxplot with Sample Size

```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm',
                color='white', showfliers=False)
medians = penguins.groupby('species')['
    flipper_length_mm'].median()
counts = penguins.groupby('species')['
    flipper_length_mm'].count()
for i, sp in enumerate(medians.index):
     plt.text(i, medians[sp]-2, f"n={counts[sp]}", ha='
         center', color='red')
plt.title("Boxplot␣with␣Sample␣Size")
```

A key disadvantage of boxplot is that it fails to show the size of each group.

Boxplot with Sample Size



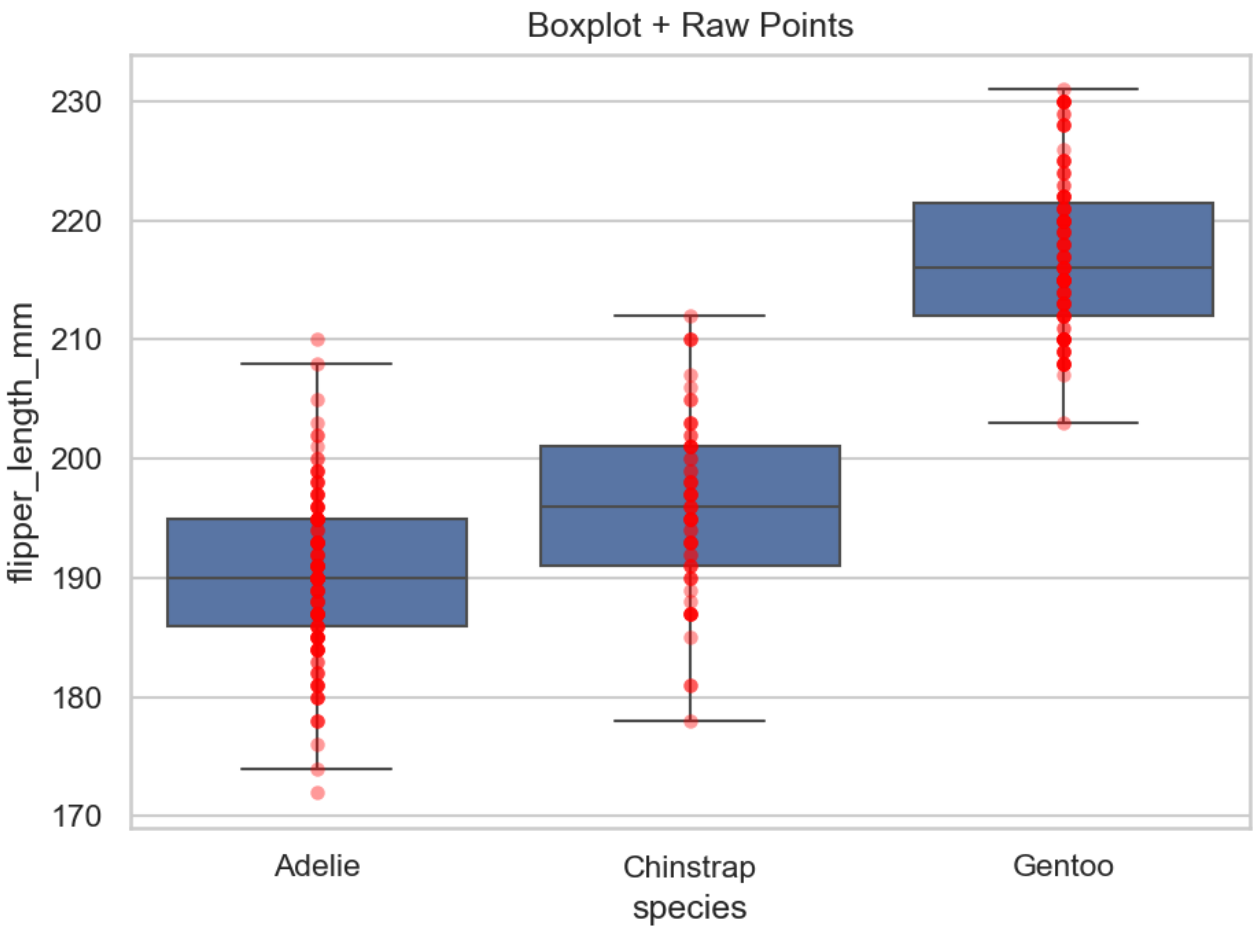
Interpretation: Annotating sample sizes gives context to the boxplot distributions.

Boxplot + Raw Points

```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm', showfliers=False)
sns.stripplot(data=penguins, x='species', y='
    flipper_length_mm', color='red', alpha=0.4)
plt.title("Boxplot + Raw Points")
```

Another way to demonstrate the size of each group is stripplot

Boxplot + Raw Points

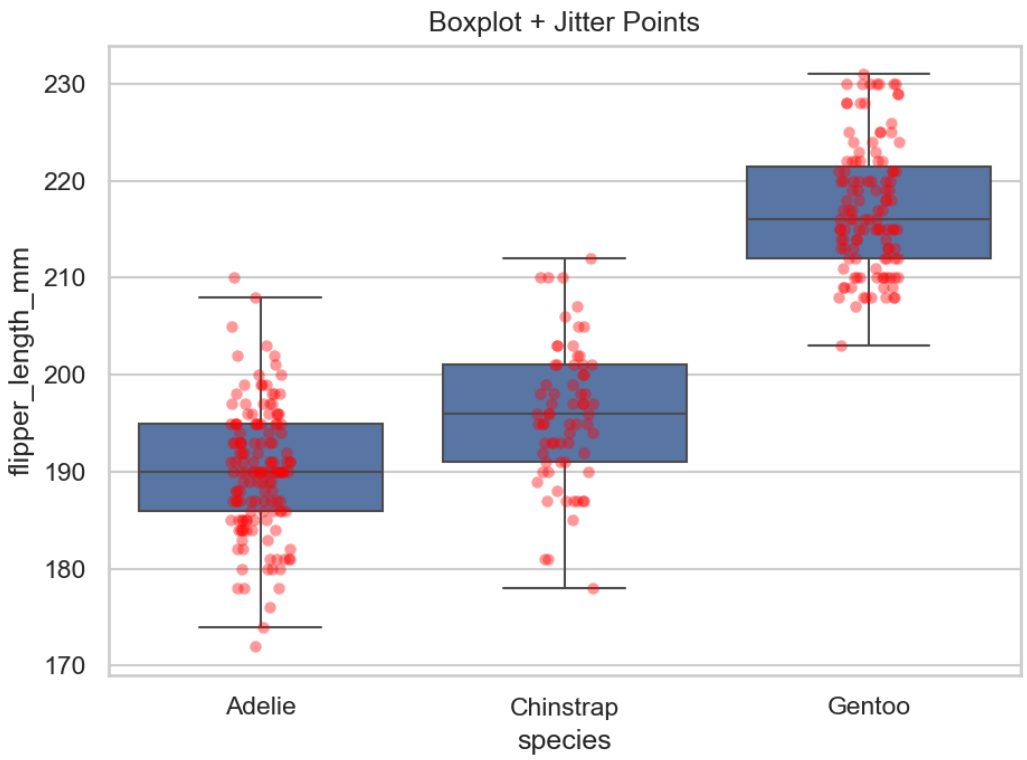


Interpretation: Overlaying raw points shows individual observations alongside summary statistics.

Boxplot + Jitter Points

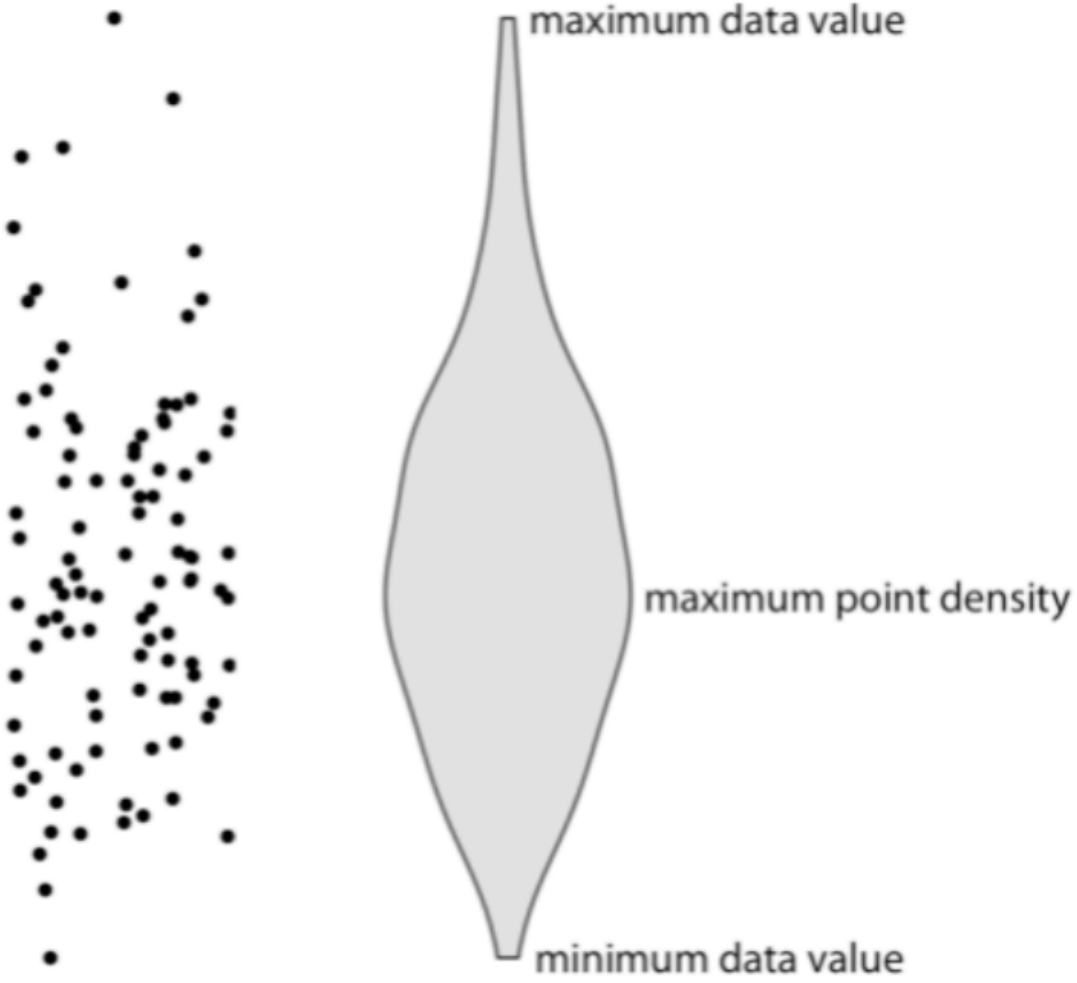
```
sns.boxplot(data=penguins, x='species', y='
    flipper_length_mm', showfliers=False)
sns.stripplot(data=penguins, x='species', y='
    flipper_length_mm', color='red', alpha=0.4, jitter=
    True)
plt.title("Boxplot + Jitter Points")
```

Boxplot + Jitter Points



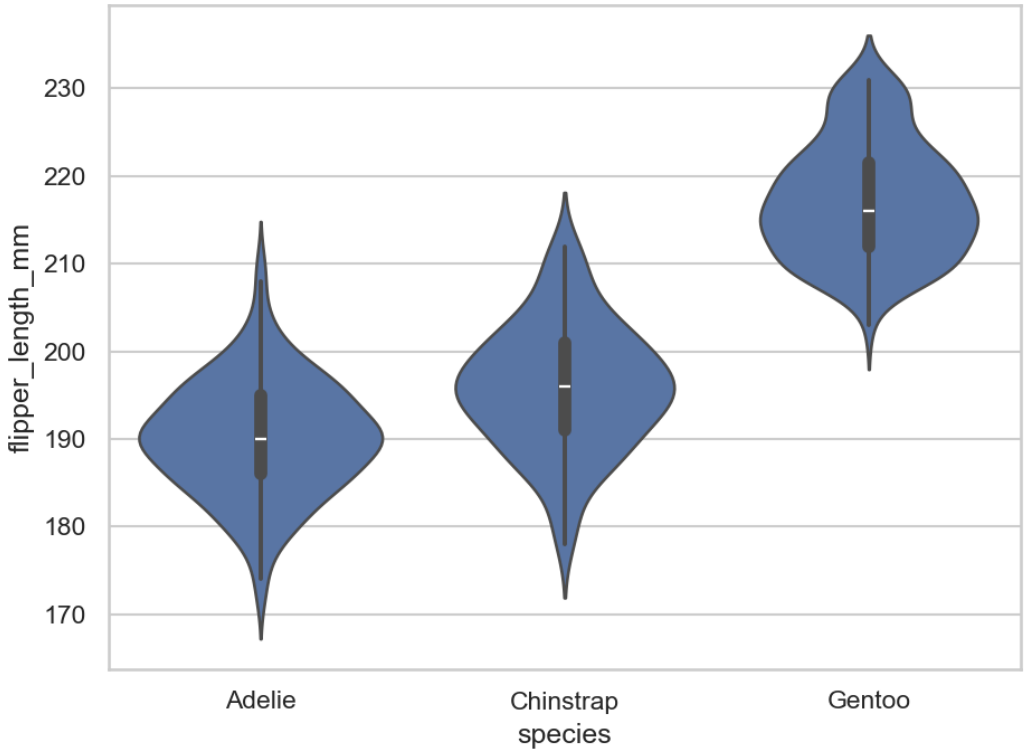
Interpretation: Adding jitter reduces overplotting, improving visualization of dense points. Can we combine the distributions of points and flipper_length_mm?

Definition of violin Plot



Violin Plot by Species

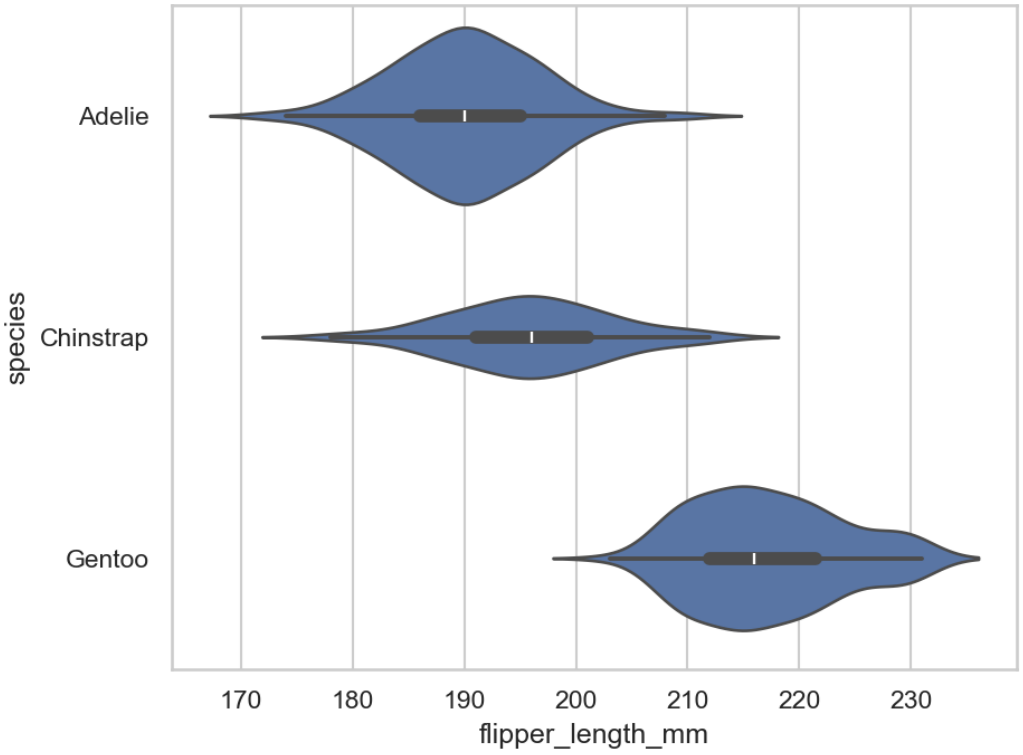
```
sns.violinplot(data=penguins, x='species', y='
    flipper_length_mm')
```



Interpretation: Violin plots combine boxplot and density, showing distribution shape per species.

Violin Plot with Scale Count

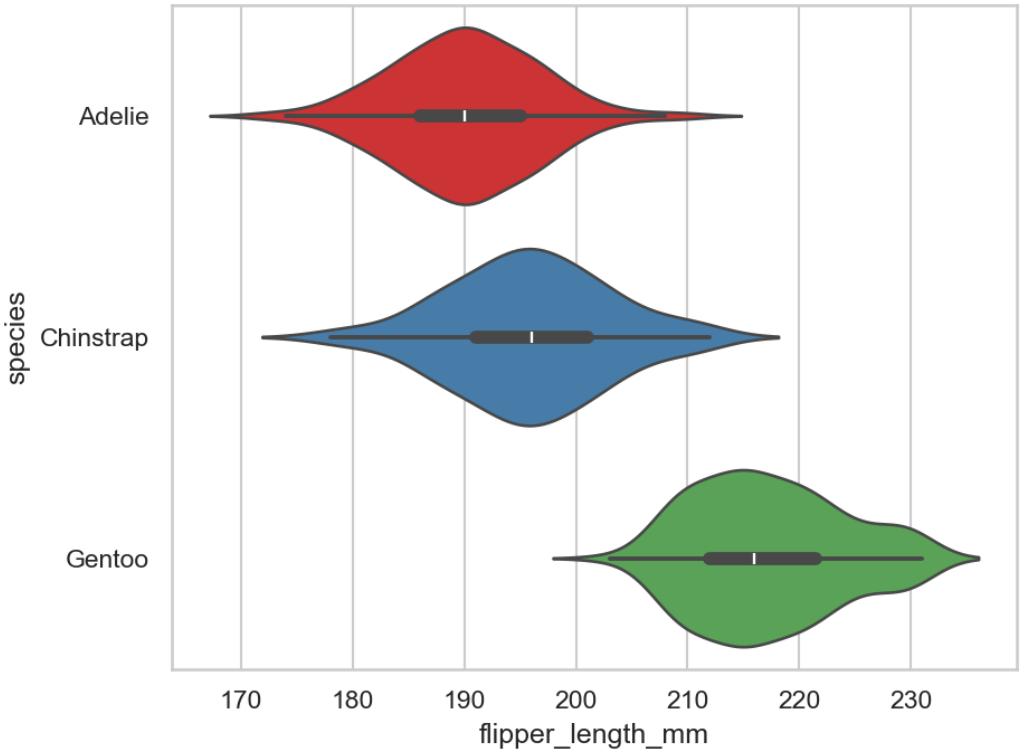
```
sns.violinplot(data=penguins, y='species', x='
    flipper_length_mm', scale='count')
```



Interpretation: Scaling by count shows species with more data points wider in the violin plot.

Violin Plot with Palette

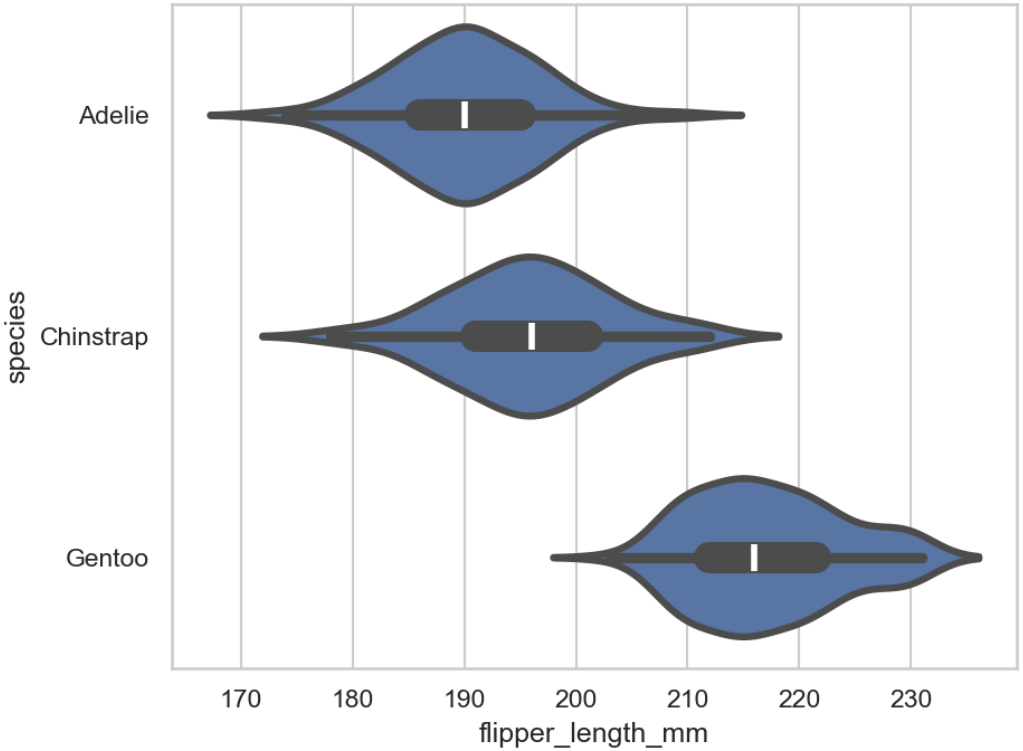
```
sns.violinplot(data=penguins, y='species', x='
    flipper_length_mm', palette='Set1')
```



Interpretation: Using a color palette improves visual differentiation between species.

Violin Plot with Line Width

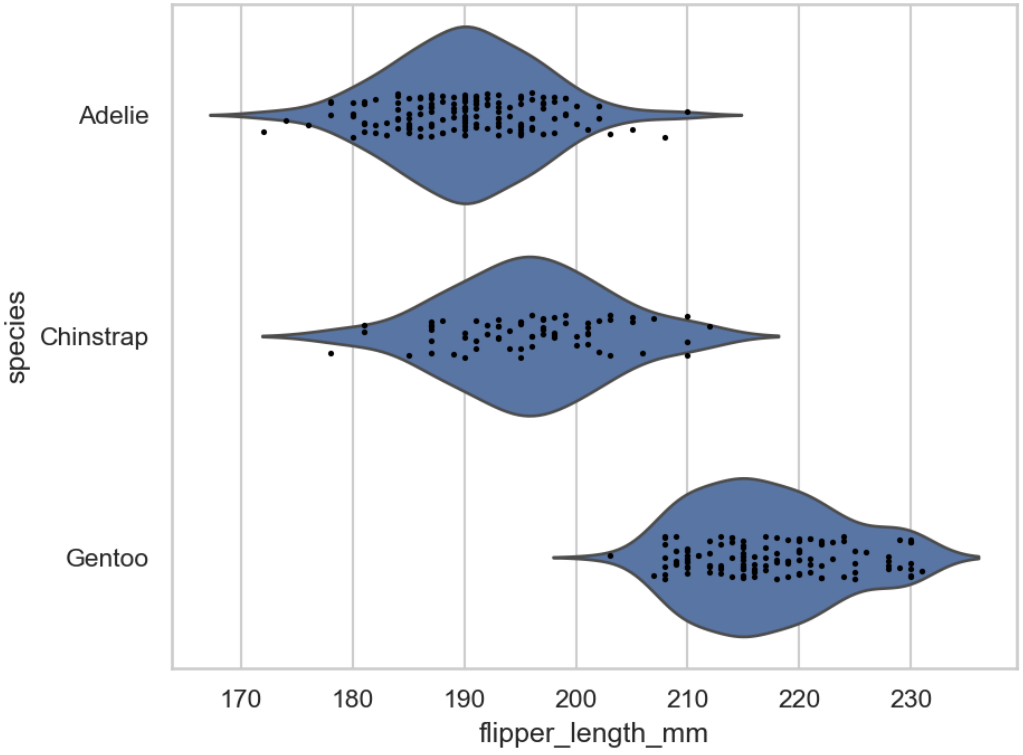
```
sns.violinplot(data=penguins, y='species', x='
    flipper_length_mm', linewidth=3)
```



Interpretation: Thicker outlines highlight the violin shapes clearly.

Violin Plot + Strip Points

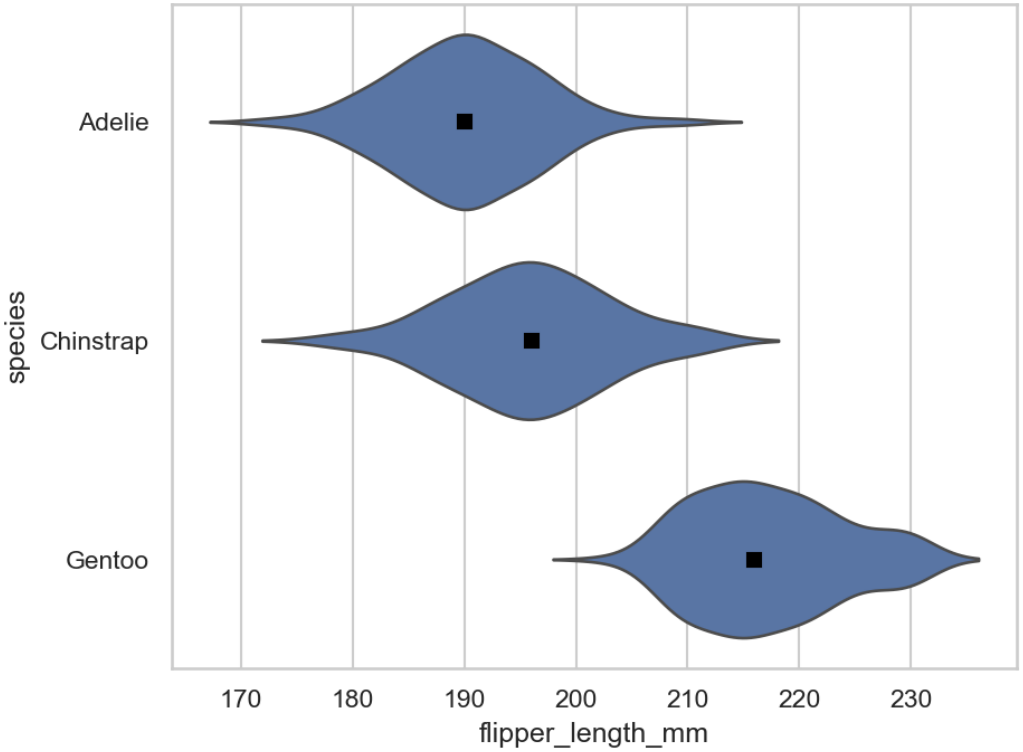
```
sns.violinplot(data=penguins, y='species', x='
    flipper_length_mm', inner=None)
sns.stripplot(data=penguins, y='species', x='
    flipper_length_mm', color='black', size=2.5, jitter
    =True)
```



Interpretation: Adding points shows raw data over the density shape

Violin Plot with Medians

```
sns.violinplot(data=penguins, y='species', x='
    flipper_length_mm', inner=None)
med = penguins.groupby("species")["flipper_length_mm"
    ].median()
for i, sp in enumerate(med.index):
    plt.plot(med[sp], i, marker='s', color='black')
```



Raincloud Plot by Sex

```
df = penguins.dropna(subset=['sex', 'body_mass_g']).
    copy()
groups = ["male", "female"]
palette = {"male": "#4E79A7", "female": "#F28E2B"}

x_grid = np.linspace(df["body_mass_g"].min(), df["
    body_mass_g"].max(), 200)
dens_dict, max_dens = {}, 0

for g in groups:
    vals = df[df["sex"] == g]["body_mass_g"].values
    kde = gaussian_kde(vals, bw_method=0.3)
    dens = kde(x_grid)
    dens_dict[g] = dens
    max_dens = max(max_dens, dens.max())

plt.figure(figsize=(6, 6))
ax = plt.gca()
```

Raincloud Plot by Sex

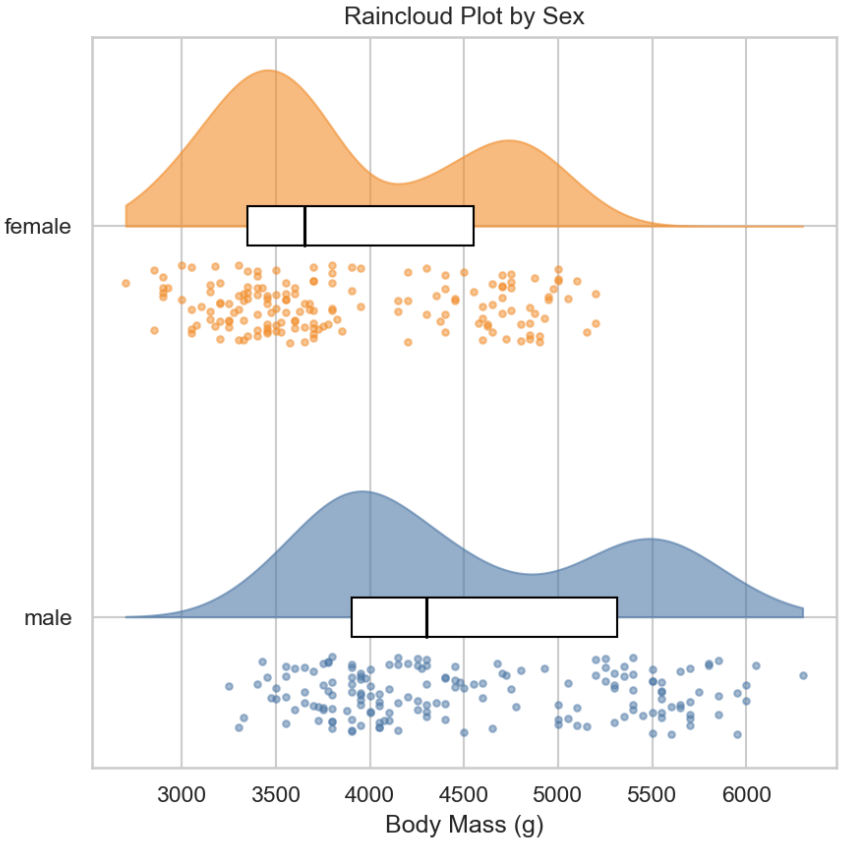
```
for i, g in enumerate(groups):
    vals = df[df["sex"] == g]["body_mass_g"].values
    dens = dens_dict[g] * (0.4 / max_dens)
    plt.fill_between(x_grid, i + dens, i, color=
        palette[g], alpha=0.6)

    q1, med, q3 = np.percentile(vals, [25, 50, 75])
    ax.add_patch(Rectangle((q1, i-0.05), q3-q1, 0.1,
        facecolor='white', edgecolor='black'))
    plt.plot([med, med], [i-0.05, i+0.05], color='
        black')

    y_jit = np.random.uniform(i-0.3, i-0.1, size=len(
        vals))
    plt.scatter(vals, y_jit, s=10, alpha=0.5, color=
        palette[g])
```

Raincloud Plot by Sex

```
# KDE + Box + Scatter
```



Interpretation: Combines density, summary statistics, and individual points for detailed distribution view.

Summary

- Explored histograms, density plots, boxplots, violin plots, and raincloud plots.
- Compared distributions by sex and species.
- Demonstrated how visualization choices (bin size, color, jitter, highlighting) affect interpretation.
- These plots provide insights for EDA and teaching visualization best practices.